Circular Dependency in Spring

Q How does the configuration class for IoC container's application context call the bean creation methods?

⇒ Spring internally creates an object of the configuration class which it manages on it's own, including invoking the @Bean methods to get / create beans.

@Configuration → internally itself an @Component!
@ComponentScan
class AppConfig {
| //...
}

Q If we autowire a dependency on a private field, why does dependency injection not fail?

```
public class OrderService {
|
|   @Autowired
|   private PaymentService payment;
|
|   // no setters defined
|
}
```

⇒ Spring's IoC container internally uses Java's Reflection API's to bypass the private access modifier on the field & able to wire the dependency!

NOTE

→ Spring can allow circular dependency to exist (even though highly discouraged), but Spring boot explicitly does not allow circular dependencies to exist

spring.main.allow-circular-reference : false
↘ by default

→ Having circular dependencies is a very bad coding practice leading to very tightly coupled code
→ It just showcases that the responsibilities are entangled, not been segrated properly, hence violating the Single Responsibility Principle (SRP)

Bean Scope

→ All beans have Singleton scope by default (only one instance created per bean definition, which is shared among the application)

prototype → each dependency leads to creation of a new instance for that bean definition

```
@Component
@Scope("prototype")
```

→ However there is a catch here
→ beans with prototype scope do not create instances at the time the IoC container is configured.
→ Rather, the first (and following) beans are created only when they are actually used somewhere in the code (lazy initialisation)

→ This in stark contrast with singleton scoped beans, which no matter whether they are used or not, get created when the

IoC container is started. (eager initialization)

Singleton confusion with @Bean (Important)

→ The term singleton used as per spring's definition might be misleading
→ It is different from the singleton design pattern

Singleton design pattern enforces that no matter wherever the object is needed or the new keyword is invoked, the same created reference must be passed.

However, spring creates one shared object per bean definition

```
@Configuration
@ComponentScan
public class AppConfig {

    @Bean
    public OrderService getOrder() {
        // ...
        return new OrderService();
    }

    @Bean
    public OrderService getOrder2() {
        // ...
        return new OrderService();
    }
    . . .
}
```

Two separate Bean Definition

→ So here, two singleton beans of OrderService will get created
⇒ In a nutshell, single object per bean definition

WARNING: Having multiple bean definitions will introduce conflict. It must be handled appropriately using @Primary / @Qualifier

When to use which scope

→ classes which hold some information are called stateful classes

```
public class User {
    private String name;
    private int age;
    :  :
}
```

→ we want our application to have multiple user's which have different state

→ so such components which are stateful should have prototype scope
→ Components like service classes, management classes which do not hold any state should have singleton scope.

Lazy & Eager Initialisation

→ Singleton's initialization is eager by default, i.e as soon as the IoC container's application context is started.

→ We can over-ride this default behaviour and make their initialization lazy, i.e. only when they are actually needed by using @Lazy annotation

```
@Component
@Lazy
public class OrderService { ...
```

→ Component's with prototype scope are lazy by default & can NOT be made eager.

Using Proxy's

```
@Component
@Lazy
public class PaymentService {

    public void pay() {
        System.out.println("payment successfull");
    }
}
```

```
@Component
public class OrderService {

    private PaymentService paymentService;
                ↳ only proxy is created
    public (@Lazy PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();  → now actual object of payment service will be created
```

```
        System.out.println("Order placed");
    }
    ...
}
```

Why Default Scope is Eager

→ Why would Spring want all the beans to get created at startup time itself? Wouldn't it crowd the application context and introduce overhead?

⇒ Yes probably, but the principle they wanted to go forward with was **Fail Fast**

↳ We want to reduce the number of bugs by checking improper / malformed code at the earliest stage & reduce unexpected behaviour and errors at runtime

∴, `spring.main.lazy-initialization = false`

↳ by default

↳ if overridden to true, all bean's will follow lazy initialization

↳ under this scenario, if you want a specific bean to be eager, use `@Lazy(value = false)` on that component

NOTE

→ Circular dependencies can be solved using Proxy's & Lazy initialisation, but it's highly recommended to re-factor the code instead